Centralized Control and Monitoring System (CCMS)

ESP32 V2.0 Firmware and System Architecture Comprehensive Report

---

1. Executive Summary

The Centralized Control and Monitoring System (CCMS) V2.0 represents a sophisticated, scalable IoT edge node designed specifically for rigorous industrial and commercial energy grid monitoring. Powered by the dual-core ESP32 microcontroller, the V2.0 firmware establishes a highly resilient architecture capable of processing complex electrical parameters, sensing structural and environmental environments, maintaining accurate asynchronous schedules, and interacting with AWS IoT Core.

Recognizing the limitations of relying purely on standard 2.4 GHz Wi-Fi for industrial deployments, the hallmark of V2.0 is the integration of a **decoupled UART expansion bridge**. This architecture enables external long-range or alternative-network transceiver cards (such as LoRa, nRF Mesh, or 802.11ah Wi-Fi HaLow) to natively intercept telemetry streams, ensuring data ingress into the cloud even in harsh RF Topographies.

---

2. Hardware Architecture & Peripheral Mapping

The ESP32 coordinates a dense array of peripherals, balancing dual UART buses, an I2C shared bus, analog digital converters (ADCs), and digital GPIOs without thread blockage.

2.1 Core Electrical & Environmental Sensors

- **Mains Voltage Sensing (Pin 35 - `MAINS_ADC_PIN`):** Utilizes the ESP32's 12-bit ADC. Tracks the presence and voltage levels of the primary AC supply via a scaled step-down conversion network.
- **Backup Battery Health (Pin 34 - `BAT_ADC_PIN`):** Constant analog sampling of the system's DC backup power supply, protecting against sudden data loss.
- **Tamper / Tilt Detection (Pin 15 - `TILT_SW_PIN`):** A mechanical tilt switch connected via an internal pull-up resistor. Actuates if a deployment cabinet is knocked over, vandalized, or physically compromised.
- **Temperature (Pin 4 - `DHT_PIN`):** A DHT11 digital interface currently recording localized chassis temperature, structurally mapped for a future industrial RTD (Resistance Temperature Detector) upgrade.
- **Actuation Relay (Pin 14 - `RELAY_PIN`):** Transistor-driven GPIO enabling the CCMS node to actively toggle external AC contactors (for lighting or breaker loads) either autonomously based on scheduled timing, or manually via remote AWS Shadow triggers.

2.2 Serial Topologies

- **Modbus Meter Link (UART 2):**
  - **Pins:** RX: 16, TX: 17
  - **Driver:** MAX485 / RS-485 level shifter.
  - **Specification:** 9600 baud, 8E1 (8 data bits, Even parity, 1 stop bit). Exclusively acts to poll standard Holding Registers from the external multi-function energy meter.
- **Remote Comms Bridge (UART 1):**
  - **Pins:** RX: 13, TX: 12
  - **Specification:** 115200 baud, 8N1. The master outgoing payload mirror for auxiliary radio cards.
- **Display & Timing (I2C Bus):**
  - **Pins:** SDA: 21, SCL: 22
  - **Devices:** Shared multi-master loop tracking a DS3231 Real-Time Clock and SSD1306 128x64 OLED Display. Address collision is physically avoided (RTC usually `0x68`, Display `0x3C`).

2.3 Human-Machine Interface (HMI)

- **Display:** Adafruit SSD1306 OLED screen rendering segmented GUI blocks for direct field-technician interaction.
- **Navigation Button (Pin 32):** Hardware-debounced tactile button used by field agents to dynamically cycle the OLED screen modes without interrupting the device's polling logic.
- **Triple-Status Diagnostics (LEDs):**
  - **LED 1 (Pin 25):** Solid green when `MAINS_ADC_PIN` exceeds a raw value of 500 (indicating healthy incoming power).
  - **LED 2 (Pin 26):** Solid blue whenever the system possesses dual-authentication (Wi-Fi associated + AWS MQTT connected).
  - **LED 3 (Pin 27):** Intermits/flashes every 25 seconds concurrent with a successful MQTT JSON payload dispatch, offering an absolute visual confirmation of packet flow.

---

3. Communication Pathways & Cloud Synergy

3.1 Primary Wi-Fi & AWS IoT Encrypted Stream

The system connects to local 802.11b/g/n networks, escalating standard TCP traffic to a fully secured mutual-TLS (mTLS) session using hard-coded private keys and certificates (`credentials/` payload) targeting an AWS IoT Core endpoint ().

- **Telemetry Stream:** Routine JSON structures (up to 1024 bytes) push natively to `meter/telemetry/Meter_001`. The backend AWS core handles rule processing to shift this into databases.
- **JSON Architecture:** The serialized payload integrates epoch timestamps, 32-bit Modbus floating point values, Battery/Mains analog interpretations, physical tilt statuses, and the DHT reading.

3.2 Extensible Comms Bridge (UART 1 - The Secondary RF Link)

To secure the physical infrastructure against Wi-Fi failures, range limits, or extreme multi-path interference common in substations, the CCMS node utilizes `Serial1` as a mirrored data bridge.

- **How it Works:** Any time the `publishMqttData()` function executes serialization for AWS, that identical `payload.c_str()` is shunted downstream across Pin 12 (TX) at 115200 baud.
- **External Integration (nRF, LoRa, Wi-Fi HaLow):** An attached third-party communication expansion board (e.g., custom LoRaWAN module, nRF24 Mesh relay) simply UART-Rx's the framed JSON data and repackets it via its specific PHY layer. This ensures that the CCMS ESP32 doesn't need to bloat its firmware with specific RF protocol stacks; it merely acts as a standardized data fountain.

---

4. Software Subsystems and Execution Threads

4.1 Chronological Synchronization (NTP + DS3231)

The CCMS guarantees sub-second timestamping accuracy even amidst rolling blackouts. Upon connecting to Wi-Fi, the core invokes an NTP packet request to `pool.ntp.org` (and `time.nist.gov`). Once a valid stratum epoch is captured (and scaled by +5.5 hours for IST), it is permanently written into the DS3231 hardware register via `rtc.adjust()`. If Wi-Fi fails entirely, internet-independent timestamping is continuously derived from the local RTC crystal.

4.2 State Twin (AWS Device Shadows)

The device subscribes structurally to the MQTT Topic: `$aws/things/Meter_001/shadow/update/delta`. When a master system administrator issues a change command on the AWS console, the local node receives a parsed `JsonObject "state"`. Known vector overrides include:

- `relay_state`: Directly forces the Relay Pin 14 High/Low immediately.
- `timeToAutoTurnOn` / `timeToAutoTurnOff`: Mutates localized global variables detailing automated schedules (e.g., 18:00 start, 06:00 kill) ensuring physical logic operates independent of live server pinging.
- `device_state`: An administrative string indicating the logical mode of operation (Active vs Maintenance mode).

4.3 Modbus Polling (IEC 61131 Compliant)

Leveraging `ModbusMaster.h`, CCMS node systematically loops over `numMqttRegs` predefined IEEE-754 endpoints (e.g., 3003, 3009). The registers are physically demanded via `node.readHoldingRegisters()`. Each read applies an inline 120ms blocking delay to respect standard RS-485 bus timing boundaries. Values are returned as dual 16-bit blocks, natively joined, byte-swapped if structurally required via `useByteSwap` logic, decoded to C++ floats, validated recursively against `NaN` and `Infinity` errors, and lastly pooled into the `mqttValues[]` global array buffer.

4.4 The Asynchronous Main Loop

The `loop()` function in `main.cpp` runs at a virtually unbounded frequency. Its structure:

1. Verify Wi-Fi topology (Auto-reconnect + Re-fire NTP).
2. Ping `mqttClient.loop()` to sustain TLS keep-alive.
3. Execute local ADC mapping (`batteryVoltage`, `mainsRaw`, `tiltSwitchState`).
4. Capture physical debounces on `BUTTON_PIN`. Rotate the `currentScreen` modal flag strictly modulo 3.
5. Render the local OLED display using current globals (0: OS stats, 1: Envo stats, 2: Load parameters).
6. Fire Modbus array reads.
7. Check `millis() - lastMqttPublish >= 25000`. If triggered, serialize JSON memory pool, flush to MQTT stream, flush to UART Bridge, and flip LED 3.

---

5. Development Dependencies & Compilations

The system is constructed under Espressif 32 (via PlatformIO CLI). Vital `lib_deps` components driving execution logic:

- **`bblanchon/ArduinoJson@^7.2.2`**: Memory-efficient serialization engine mapping the vast array outputs into pure UTF-8 JSON templates.
- **`knolleary/PubSubClient@^2.8`**: Thread-safe MQTT client wrapper framing standard QoS messages to AWS.
- **`4-20ma/ModbusMaster@^2.0.1`**: Highly reliable protocol driver decoding complex sequential framing via UART2.
- **`adafruit/RTClib@^2.1.3`**: Simplifies standard I2C register mathematics shifting UNIX timescales natively into the DS3231 module.
- **`adafruit/Adafruit SSD1306@^2.5.9`** + **`GFX Library`**: Memory-mapped OLED graphics handler preventing pixel burn and ghosting.

6. Forward Progression

CCMS V2.0 establishes an impenetrable architecture baseline for true industrial monitoring. Integrating the offline RTC, real-time AWS shadow handling, comprehensive multi-tier environmental logging, and the heavily modular UART1 communications bridge ensures the system is practically future-proof. Whether scaled into Wi-Fi heavy infrastructure or relegated into dense concrete zones requesting Sub-GHz routing, the core intelligence remains static, pristine, and wholly isolated.